Design and Implementation of an Autonomous Target-Seeking Robot "Mini Edger Guapo"

ECE118 – Sluggers of the Lost Goal

Team 32
Jair Orellana
Gavin Wang
Braden Monroe

June 13, 2026

Contents

1 Executive Summary **3**

2 Objectives **3**

3 System Overview **3**
3.1 Hardware Used . . . 3
3.2 Software Architecture . . . 4

4 Design Approach **4**
4.1 Mechanical - CAD Design . . . 4
4.1.1 Base Layer . . . 4
4.1.2 Shooting Layer . . . 5
4.1.3 Basket . . . 6
4.2 Electrical . . . 7
4.2.1 Track Wire . . . 7
4.2.2 Beacon Detector . . . 7
4.3 High-Level Behavior . . . 9
4.4 **Setup Sub-State Machine** . . . 9
4.4.1 Spin Search State . . . 9
4.4.2 Beacon Found State . . . 9
4.4.3 Drive to Tape State . . . 9
4.4.4 Turning Perpendicular State . . . 10
4.5 **Tape Following Sub-State Machine** . . . 10
4.5.1 Localize State . . . 10
4.5.2 Align Straight State . . . 10
4.5.3 Corner Detection and Handling . . . 10

4.5.4 ISZ State . . . 10
4.6 Obstacle Avoidance Sub-State Machine . . . 10
4.6.1 Backup State . . . 10
4.6.2 Turn and Traverse States . . . 10
4.6.3 Recovery and Realignment . . . 11
4.6.4 Return to Tape State . . . 11
4.7 **Shooting Sub-State Machine** . . . 11
4.7.1 Centering State . . . 11
4.7.2 Center Forward State . . . 11
4.7.3 Beacon Search State . . . 11
4.7.4 Jerk (Alignment) State . . . 11
4.7.5 Shoot State . . . 12
4.7.6 Blind Firing States . . . 12
4.8 State Machine Overview . . . 12

5 Event Checker and Motor Abstraction Layer **12**
5.1 Design Overview . . . 12
5.2 Beacon Detection with Filtering and Hysteresis . . . 12
5.3 Tape Sensor Event Filtering (Debounce Logic) . . . 13
5.4 Bumper Debounce Filtering . . . 14
5.5 Trackwire Filtering . . . 14
5.6 Motor Control Abstraction Layer . . . 14
5.7 System Integration Impact . . . 15

6 Testing and Validation **15**
6.1 Test Plan . . . 15
6.2 Results . . . 15
6.3 Problems Encountered . . . 16

7 Performance Analysis **16**
7.1 Demonstration Video . . . 17

8 Future Improvements **17**

9 Conclusion **17**

1 Executive Summary

For the Final Project, an autonomous robot was built to find a target and shoot ping pong balls at it while avoiding obstacles. By using intended hardware for proper obstacle avoidance, target detection, and State Machine Software, our robot was able to accomplish these tasks. The final result of our robot was a functional autonomous platform capable of locating and engaging the target while avoiding most obstacles, though improvements in sensor accuracy and targeting consistency could further enhance performance.

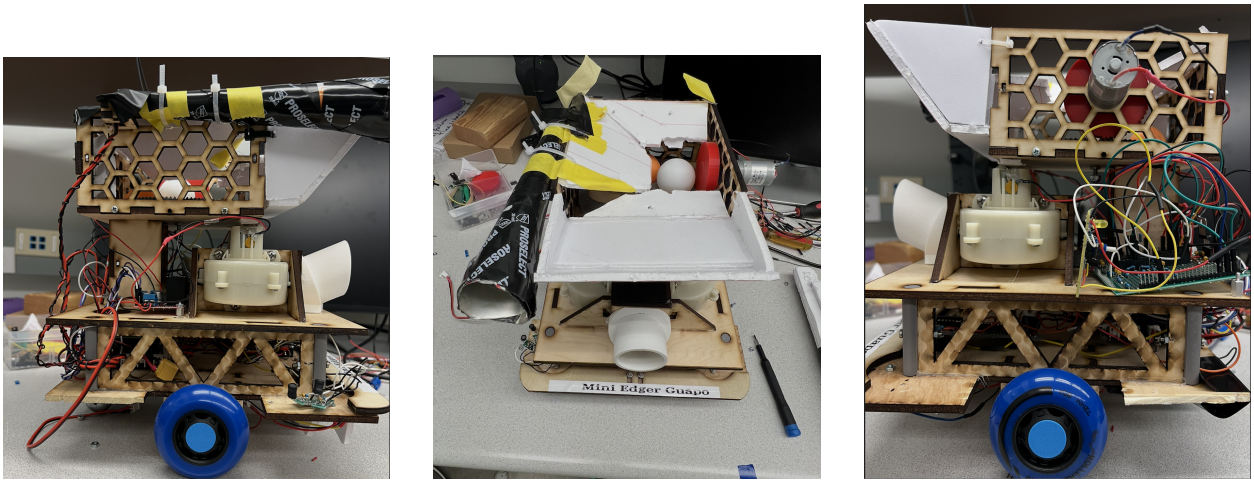


Figure 1: Final autonomous robot

2 Objectives

- Implement hierarchical state machine navigation
- Integrate sensors with Events & Services framework
- Validate robot performance through testing
- Improve system consistency and functionality

3 System Overview

3.1 Hardware Used

- UNO32 Board with I/O Shield
- DC Motors and H-Bridge
- Tape Sensors (Boundary Detection)
- Track Wire Sensor (Navigation)
- Beacon Detector (Goal Finding)
- Micro Limit (Bumper) Switches (Obstacle Detection)

3.2 Software Architecture

The robot software is built using the Events & Services framework. Behavior is implemented using a hierarchical state machine (HSM).

4 Design Approach

4.1 Mechanical - CAD Design

4.1.1 Base Layer

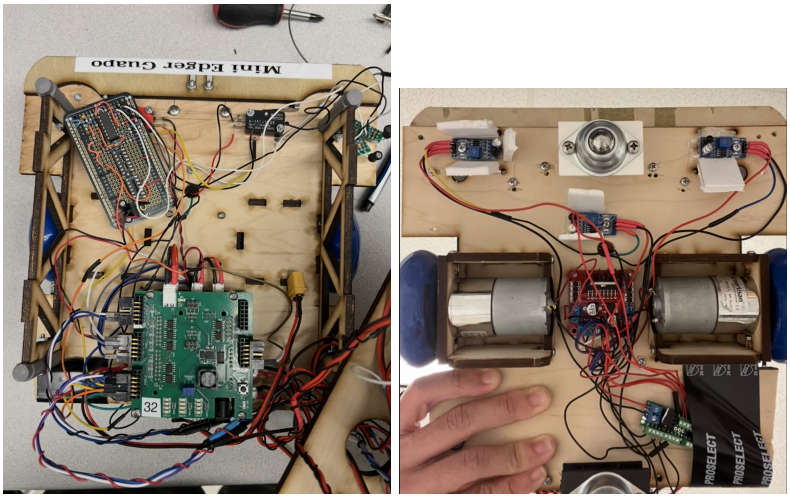


Figure 2: Final autonomous robot developed for the ECE 118 competition.

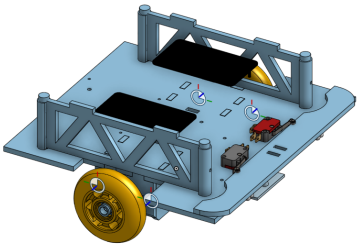


Figure 3: CAD for base layer

The first part of the robot consists of the base layer, which covers sensing, navigation, and movement. This is also where the UNO stack, the battery, and most of the main components are housed.

The major sensing components on this layer are the bumper, tape sensors, and track wire sensor. The tape sensors provide digital I/O signals to the UNO stack to help with line following, while the bumper and track wire sensors work in tandem if and when obstacles are found.

Other components include the H-bridge and DC motors that move the robot around and the roller bearings that reduce wobble and increase stability while in motion. There is also a 3.3 voltage regulated array on this layer to power all these sensors.

4.1.2 Shooting Layer

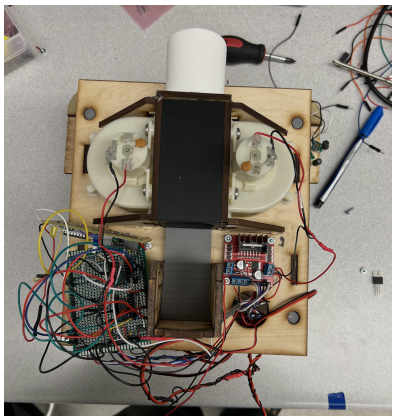


Figure 4: Final autonomous robot developed for the ECE 118 competition.

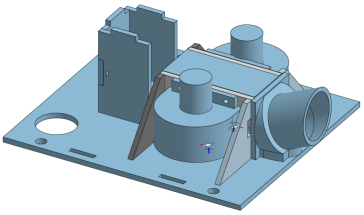


Figure 5: CAD for shooting layer

The middle section of the robot is relatively simple, containing the shooter and the beacon detector circuit. The shooter uses a 3D-printed housing to hold together the flywheels and exit tube, all of which also slot into the plate for this layer. Near the bottom of Figure 3, you can also see the gray ramp and plywood that hold the basket up and help funnel dispensed balls into the shooter.

The majority of the beacon detector circuit can be seen on the bottom left side of Figure 3. This is all logic other than the phototransistor, which has been routed to the side of the basket to enable stronger signal reading, and the digital I/O, which is ported to the UNO stack.

4.1.3 Basket

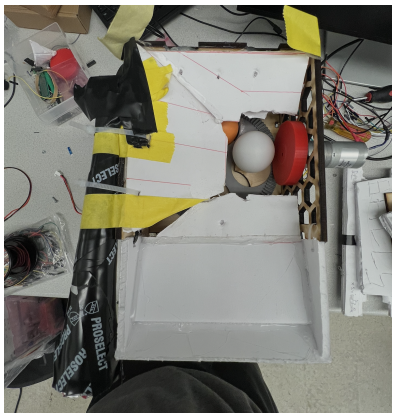


Figure 6: Final autonomous robot developed for the ECE 118 competition.

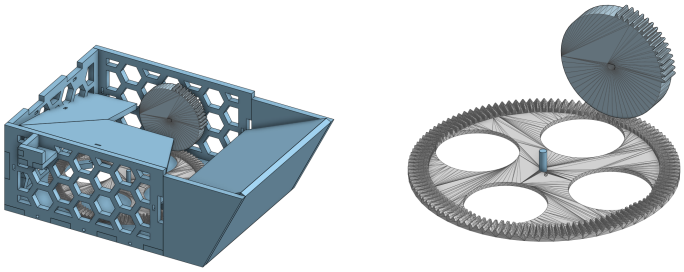


Figure 7: CAD for full basket, gear array

The final part of the robot consists of a mixed media storage and feeding system. The main frame is made of laser cut plywood, with foam core board acting as a funnel and storage expansion. The dispensing system is built from a DC motor and two gears. Additionally, for beacon detection, the phototransistor and cone are attached to the side of the basket.

The ingenuity behind the feeder design is the oscillation achieved between closed and open states, despite only being run on a DC motor. Mathematically, the cylindrical gear and the face gear have a 2:1 gear ratio, and the red cylindrical gear also has teeth on 1/4th of its body. Together, this means the face gear turns 1/8th of its total revolution every time the red gear turns once. Due to the arrangement of the 4 ping pong sized-holes in the face gear, this means that the ball feeder can be run continuously while only ever letting one ball through at a time. In addition to this, our anti-jam tactics are simple, and include reversing the direction of this gear rotation just for a moment, as this allows any balls pinched between the face gear and the side of the drop chute to be released.

4.2 Electrical

4.2.1 Track Wire

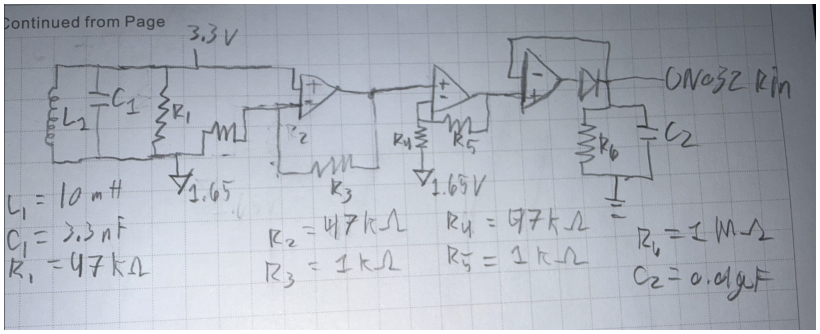


Figure 8: Trackwire schematic consists of one tank circuit, two gain stages of 47 (non-inverting op amp), and a peak detector.

The Trackwire circuit consists of an RLC circuit, two non-inverting op amps, and peak detector in order for the system to interact with the UNO32 Microcontroller. The purpose was to detect the wire around the obstacle and hold a peak voltage until the obstacle was avoided and no signal was left. Since the RLC circuit only detects and has a very small signal, two gain stages of 47 were needed in order to amplify the signal and get a noticeable difference between the obstacle being sensed, and when it wasn't.

4.2.2 Beacon Detector

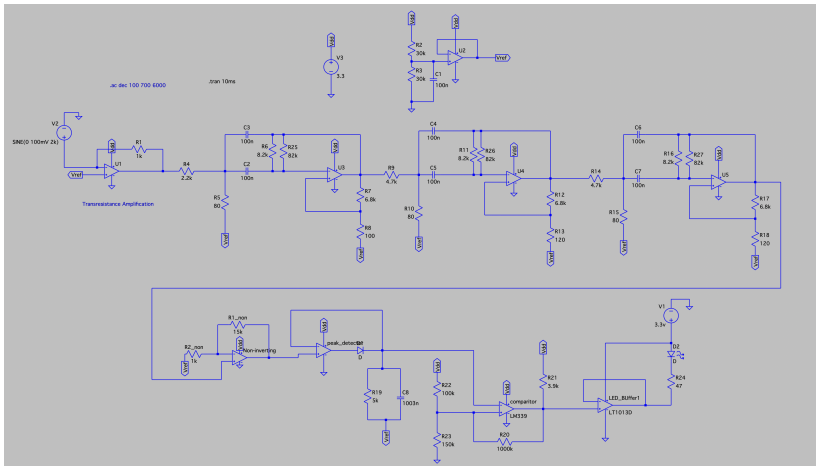
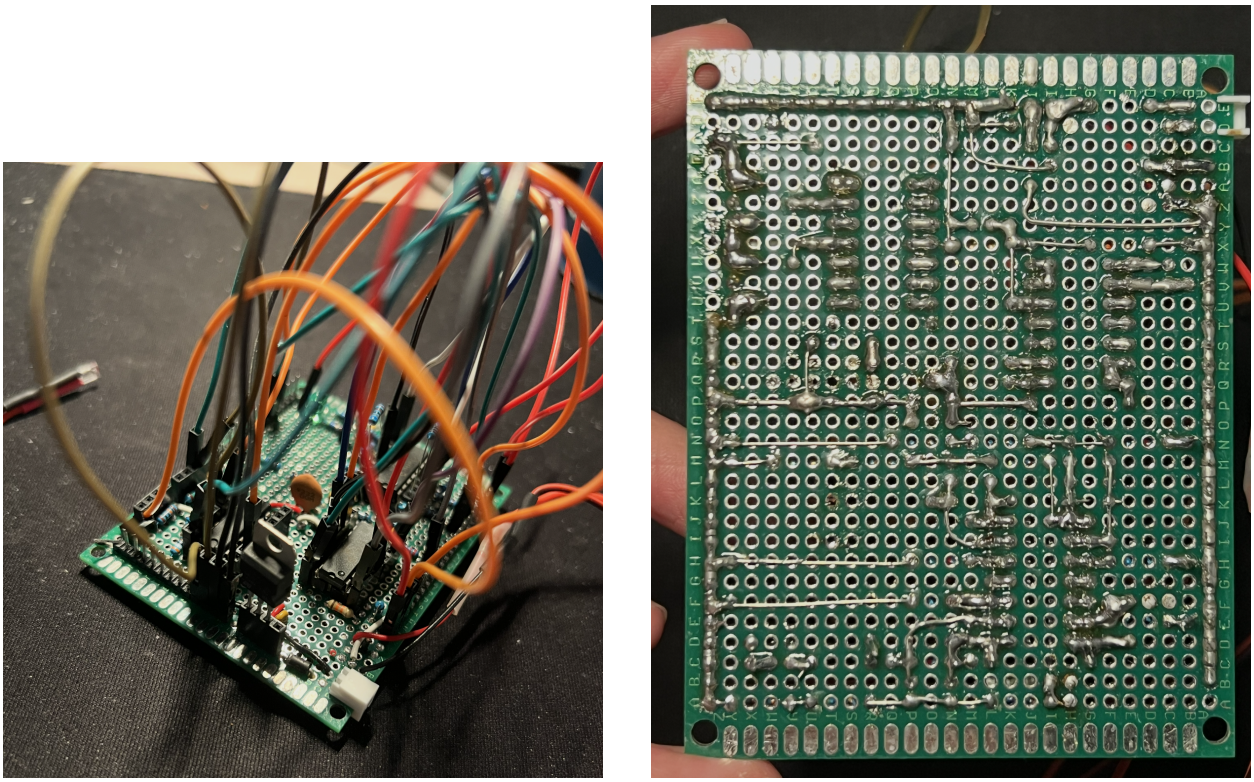


Figure 9: Beacon detector schematic consisting of a three-stage Butterworth Sallen-Key band-pass filter, peak detector, and comparator.

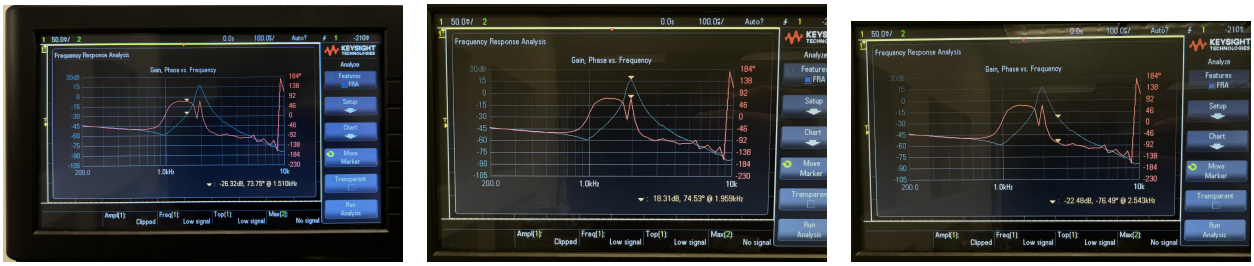
The beacon detector consists of a three-stage Butterworth Sallen-Key band-pass filter followed by a peak detector and comparator circuit. The system is designed to detect the 2 kHz competition beacon while attenuating signals outside the target frequency range. A Butterworth response was selected due to its flat passband and predictable frequency behavior.



(a) Top view (b) Bottom view

Figure 10: Fabricated beacon detector circuit on perfboard.

The filter was designed to peak at 2 kHz. Figure 11 shows the measured response at 1.5 kHz, 2.0 kHz, and 2.5 kHz. As expected, the 2 kHz signal experiences the highest gain, while frequencies above and below are attenuated.



(a) 1.5 kHz (b) 2.0 kHz (c) 2.5 kHz

Figure 11: Measured frequency response of the beacon detector.

During testing, component tolerances shifted the filter center frequency from 2 kHz to approximately 1.85 kHz due to capacitor variation (±10%). To correct this, an 82 kΩ resistor was placed in parallel with the existing 8.2 kΩ resistor, tuning the response back to approximately 2 kHz.

The filtered signal is then processed using a peak detector that converts the AC waveform into a DC voltage for digital detection. Initial testing with a 10 kΩ discharge resistor resulted in a slow decay, causing the output to remain high after the beacon was removed. Reducing the resistor to 5 kΩ improved response time. With a 1003 nF capacitor, the time constant is:

$$\tau = RC = (5\,\text{k}\Omega)(1003\,\text{nF}) \approx 5.0\,\text{ms}$$

Finally, the comparator converts the analog signal into a digital output. Thresholds were tuned to 1.9 V and 2.0 V to ensure reliable detection while minimizing false triggering from noise and ambient infrared interference.

4.3 High-Level Behavior

The robot behavior is implemented using a hierarchical state machine (HSM) based on the Events and Services framework. The top-level state machine coordinates four main operational modes: initialization, setup navigation, tape following, obstacle avoidance, and shooting.

After startup, the system enters an **Idle** state where all subsystems are initialized and the robot remains stationary for a short delay. This ensures all sensors and sub-state machines are properly configured before motion begins.

The robot then transitions through the following phases:

- **Setup Phase:** Localizes the robot and navigates toward the Initial Shooting Zone (ISZ).
- **Tape Following Phase:** Follows boundary tape using the tape-following sub-state machine.
- **Obstacle Avoidance Phase:** Handles bumper and trackwire events to navigate around obstacles.
- **Shooting Phase:** Engages beacon detection, alignment, and firing once the ISZ is reached.

All transitions are event-driven. Tape detection triggers navigation behavior, reaching the ISZ triggers the shooting routine, and bumper events temporarily interrupt navigation for obstacle recovery.

4.4 Setup Sub-State Machine

The Setup Sub-HSM is responsible for localizing the robot, acquiring the beacon signal, and aligning with the boundary tape before entering the main navigation phase.

4.4.1 Spin Search State

The robot rotates in place while scanning for a valid beacon signal. Multiple detections are required to reduce false positives. Once a stable beacon reading is confirmed, the robot transitions to the Beacon Found state.

4.4.2 Beacon Found State

After detecting the beacon, the robot performs a short reverse maneuver to create clearance and improve alignment stability. This is controlled using a fixed timer before transitioning to tape acquisition.

4.4.3 Drive to Tape State

The robot drives forward while monitoring tape sensors. Upon detecting the boundary tape, it transitions into a corrective alignment phase.

4.4.4 Turning Perpendicular State

The robot performs a controlled turn to align perpendicular to the tape boundary. Once alignment is complete, it signals the top-level controller that the Initial Shooting Zone (ISZ) has been reached.

4.5 Tape Following Sub-State Machine

The Tape Following Sub-HSM maintains alignment along the boundary tape and detects corners to track progress around the field.

4.5.1 Localize State

The robot follows the tape using differential motor control. Center tape detection is used as the primary feedback signal. Corner detection is temporarily suppressed during early navigation to avoid false triggers.

4.5.2 Align Straight State

This state corrects heading errors by adjusting motor speeds until the tape is centered under the sensors. Once alignment is achieved, the robot resumes forward tracking.

4.5.3 Corner Detection and Handling

Corner events are detected when tape continuity is interrupted. Each valid corner increments a counter. After four corners, the robot assumes it has completed a full loop and reached the ISZ.

For intermediate corners, the robot executes a timed turning maneuver before resuming tape following.

4.5.4 ISZ State

After completing two corners, the robot stops motion and sends an event to the top-level state machine indicating arrival at the ISZ. This triggers the transition into the Shooting phase.

4.6 Obstacle Avoidance Sub-State Machine

The Obstacle Avoidance Sub-HSM handles bumper-triggered interruptions and guides the robot around obstacles while preserving navigation progress.

4.6.1 Backup State

The robot reverses briefly to create clearance from the obstacle. A timer controls the duration of this maneuver.

4.6.2 Turn and Traverse States

The robot executes a 90-degree turn followed by forward motion along the obstacle boundary. Timers regulate movement distance, and bumper feedback is monitored for additional collisions.

4.6.3 Recovery and Realignment

If additional collisions occur, the system re-enters the backup routine and increases recovery intensity. Once the obstacle is cleared, the robot re-aligns with the tape.

4.6.4 Return to Tape State

The robot drives forward until the boundary tape is reacquired, then resets obstacle counters and resumes normal tape following.

There were several edge cases that needed to be handled in the Obstacle Avoidance (OA) Sub-HSM to ensure reliable navigation around obstacles:

- **Staircase re-entry issue:** After the initial right turn, a bumper event could cause the system to re-enter the OA Sub-HSM and reinitialize the avoidance sequence. This could lead to repeated partial obstacle traversal and oscillatory behavior.
- **Blocked second right turn:** During the final re-alignment back toward the tape, if another obstacle blocked the path, the robot could become stuck in the avoidance routine.

  To handle this, the robot performs a short backup followed by a slight right pivot. This helps force clearance and allows the robot to continue progressing toward the tape.

4.7 Shooting Sub-State Machine

The Shooting Sub-HSM is responsible for beacon localization, targeting, and projectile firing once the robot reaches the ISZ. It includes both vision-based and blind fallback strategies.

4.7.1 Centering State

The robot performs a controlled turn-and-drive maneuver to position itself near the center of the shooting zone. Timers are used to regulate both rotation and forward travel.

4.7.2 Center Forward State

A final alignment maneuver is performed to orient the robot toward the expected beacon direction before initiating search behavior.

4.7.3 Beacon Search State

The robot rotates in place while scanning for the beacon signal. A timeout is used to prevent indefinite searching. If the beacon is not found within the time limit, the system transitions to blind firing mode.

When the beacon is detected, the robot records the signal strength and transitions to a fine alignment phase.

4.7.4 Jerk (Alignment) State

A short corrective motion is performed to align the robot precisely with the beacon. This ensures optimal firing direction before engaging the shooter system.

4.7.5 Shoot State

The flywheel is spun up to operating speed, and the dispensing mechanism feeds projectiles. The system includes periodic unjamming routines and a reset timer that returns the robot to beacon search after a fixed duration.

4.7.6 Blind Firing States

If the beacon cannot be detected, the robot enters a blind firing sequence:

- **Blind Pivot:** Rotate to a predefined orientation.
- **Blind Shoot:** Fire while spinning up the flywheel.
- **Blind Pivot Back:** Reorient for a second firing position.
- **Blind Shoot Again:** Execute a secondary firing attempt.

This fallback strategy ensures projectile firing even under poor sensor conditions.

4.8 State Machine Overview

5 Event Checker and Motor Abstraction Layer

The event checker module and motor control layer form the low-level interface between hardware sensors and the hierarchical state machine. Their role is to filter noisy sensor data, generate clean transition events, and abstract motor control into simple commands for behavior-level code.

5.1 Design Overview

The system was designed with three main goals:

- Noise reduction through filtering and hysteresis
- Event generation only on stable transitions
- Separation of sensing, decision-making, and actuation

5.2 Beacon Detection with Filtering and Hysteresis

The beacon detector uses a moving average filter combined with a hysteresis-based state machine. Raw ADC values are first smoothed before being evaluated against ON/OFF thresholds.

Filtered Beacon Signal The beacon signal is processed using a circular buffer average filter:

```
uint16_t Beacon_ReadFiltered(void)
{
    uint16_t newSample = Beacon_ReadRaw();

    BeaconSum -= BeaconSamples[BeaconIndex];
    BeaconSamples[BeaconIndex] = newSample;
    BeaconSum += newSample;
```

```
    BeaconIndex = (BeaconIndex + 1) % BEACON_FILTER_SIZE;

    return (BeaconSum / BEACON_FILTER_SIZE);
}
```

This reduces high-frequency noise and smooths transient spikes in the infrared beacon signal.

Event Generation with Hysteresis Beacon detection uses a two-threshold system with a confirmation counter:

```
if (!beaconDetected)
{
    if (val > BEACON_ON_THRESHOLD)
    {
        detectCount++;

        if (detectCount >= 10)
        {
            beaconDetected = TRUE;

            thisEvent.EventType = BEACON_DETECTED;
            thisEvent.EventParam = val;
            PostTopHSM(thisEvent);
        }
    }
    else
    {
        detectCount = 0;
    }
}
```

This prevents false triggers caused by reflections, noise, or partial occlusion during motion.

5.3 Tape Sensor Event Filtering (Debounce Logic)

Tape sensors use a debounce counter to ensure that a valid tape detection only occurs after multiple consistent readings.

```
if (CTS_Reading == 1)
{
    if (cts_onCount < CTS_DEBOUNCE_TICKS)
    {
        cts_onCount++;
    }

    if (cts_onCount >= CTS_DEBOUNCE_TICKS)
    {
        curEvent = CTS_ON_TAPE;
    }
}
else
{
    cts_onCount = 0;
    curEvent = CTS_OFF_TAPE;
```

```
}
```

This ensures that brief reflections or edge noise do not trigger false tape transitions.

5.4 Bumper Debounce Filtering

The bumper system uses a shift-register style history buffer to detect stable presses and releases.

```
Bumpers[i].history =
    ((Bumpers[i].history << 1) | raw[i])
    & ((1 << DEBOUNCE_SAMPLES) - 1);

if (Bumpers[i].history == FULL_ON && stableState == 0)
{
    stableState = 1;
    PostTopHSM(LEFT_BUMPER_PRESSED);
}
else if (Bumpers[i].history == 0 && stableState == 1)
{
    stableState = 0;
    PostTopHSM(LEFT_BUMPER_RELEASED);
}
```

This approach eliminates mechanical bounce and ensures one event per physical press.

5.5 Trackwire Filtering

The trackwire sensor uses threshold-based debounce counting to confirm detection:

```
if (Left_Trackwire > LTW_THRESHOLD)
{
    TW_Count++;

    if (TW_Count >= TW_DEBOUNCE_TICKS)
    {
        curEvent = ON_TRACKWIRE;
    }
}
else
{
    TW_Count = 0;
    curEvent = OFF_TRACKWIRE;
}
```

This ensures that electromagnetic noise does not trigger false boundary detections.

5.6 Motor Control Abstraction Layer

Motor control is abstracted into simple speed commands for each subsystem.

```
void Motors_SetSpeed(int leftSpeed, int rightSpeed)
{
    Motors_SetLeftSpeed(leftSpeed);
    Motors_SetRightSpeed(rightSpeed);
}
```

This allows higher-level behaviors to use simple motion primitives such as forward motion and in-place rotation without dealing with PWM or GPIO logic.

5.7 System Integration Impact

By combining filtered event checkers with a clean motor abstraction layer, the system achieves:

- Stable event-driven state transitions
- Reduced sensitivity to sensor noise
- Predictable and repeatable motor behavior
- Clear separation between perception and actuation

This modular structure significantly improves debugging, since sensor behavior, event generation, and motor output can each be tested independently.

6 Testing and Validation

6.1 Test Plan

To validate the robot's functionality, each subsystem was tested independently before being integrated into the complete system. Individual testing allowed faults to be isolated and corrected more efficiently while verifying that each subsystem met its intended functionality. The primary subsystems evaluated were:

- Tape Following
- Obstacle Avoidance
- Beacon Detection and Shooting

Following subsystem validation, full-system tests were conducted to evaluate the robot's ability to autonomously locate the beacon, navigate the course, avoid obstacles, reach the ISZ, and accurately shoot the balls.

6.2 Results

Testing revealed several issues that were addressed through iterative hardware and software improvements:

- Multiple edge cases were identified in the obstacle avoidance algorithm, particularly when navigating closely spaced or staircase-pattern obstacles. Additional logic was implemented to improve recovery and path planning.
- The original shooter nozzle produced inconsistent ball trajectories. The nozzle was redesigned to improve shot repeatability and overall shooting accuracy.
- Ball jamming frequently occurred when balls were stacked directly on top of one another. The storage mechanism was redesigned into a two-layer compartment with improved funneling, resulting in significantly more reliable ball feeding.

- During software debugging, blocking code within the state machine prevented important events from being processed. This issue was resolved by creating timer-driven states that could only be entered and exited through timer events, allowing other events to continue being handled correctly by the event-driven framework.

6.3 Problems Encountered

There were multiple problems we encountered. Our mechanical flywheel basket and shooting design had some flaws such as ball jamming and overshooting. These problems were easily solved by reprinting the barrel with a different angle to lower the shooting angle and readjust the speed at which our geneva wheel was moving at. Another thing we did to prevent ping pong balls from getting stuck was to have the motor reverse the direction for a second and then continue running the normal direction for 5 seconds. Other mechanical problems was faulty whole measuring. The counteract this problem, we just drilled new holes into the chassis to fit the microcontroller and other sensors. When it came to electrical issues, our tape sensors had to be recalibrated and beacon detector was detecting the 2khz frequency in every direction when we changed from the bigger rooms to the TA room. Luckily, these problems were easier to solve. We ended up shielding the tape sensors with foam core and made a cone for the beacon detector to shield it better so it detects the correct frequency.

7 Performance Analysis

The performance and reliability of our robot improved significantly throughout the development process. While the primary state machine successfully completed the intended sequence of locating the beacon, following the tape, navigating to the Identification and Scoring Zone (ISZ), and shooting, extensive testing revealed several edge cases that required additional logic to ensure consistent operation.

The most significant improvements were made to the obstacle avoidance behavior:

- **Closely Spaced Obstacles:** During testing, we observed that if the robot encountered another obstacle immediately after completing its 90-degree right turn, its heading would become misaligned, preventing it from returning to the tape. To address this, we added a small corrective pivot whenever the robot detected a bumper event during the right turn. This allowed the robot to adjust its orientation in real time before continuing the obstacle avoidance sequence.

- **Staircase Obstacle Pattern:** A staircase arrangement of obstacles caused the obstacle avoidance state machine to re-enter while retaining state information from the previous obstacle. This occasionally resulted in incorrect behavior and missed tape detections. To solve this issue, the obstacle avoidance sub-state machine was modified to fully reset its internal counters and variables each time it was re-entered. Additionally, the middle tape sensor was temporarily ignored after detecting multiple consecutive obstacles, preventing false tape detections while navigating the staircase pattern. “‘

Another challenge involved the robot's shooting accuracy. During testing, one flywheel motor consistently produced more force than the other, causing the balls to drift to one side after leaving the launcher. Rather than redesigning the shooter, we introduced a small corrective ”jerk” immediately after aligning with the beacon in the ISZ. This slight adjustment compensated for the imbalance between the flywheels and significantly improved shot consistency.

Overall, iterative testing and refinement greatly improved the robustness of the robot. By identifying and addressing numerous edge cases throughout development, the final system was able to operate reliably under a wide variety of conditions. Watching the robot evolve from an inconsistent prototype into a dependable autonomous system demonstrated the value of systematic testing, debugging, and incremental design improvements.

7.1 Demonstration Video

A demonstration of the robot's performance can be viewed at the following YouTube playlist:
Robot Demonstration Playlist

8 Future Improvements

Although the final robot successfully completed the required tasks, several improvements could further increase its performance, reliability, and overall efficiency.

- **Improve Shooting Accuracy:** Redesign or better calibrate the flywheel mechanism to eliminate ball drift and produce more consistent shots without requiring software compensation.
- **Increase Obstacle Avoidance Robustness:** Reduce the dependence on fixed timing values and motor tuning by incorporating sensor-based feedback or closed-loop control. This would allow the robot to perform consistently despite variations caused by battery voltage or motor wear.
- **Reduce Robot Size:** Optimize the mechanical layout to create a more compact design. A smaller robot would be more maneuverable and better suited for navigating tight spaces and obstacles.
- **Increase Ball Storage Capacity:** Redesign the hopper and funneling mechanism to hold more balls while improving the reliability of ball feeding into the shooter. This would reduce reload frequency and improve scoring efficiency.

9 Conclusion

The final robot design was able to integrate mechanical, hardware, and software all together in order to achieve autonomous behavior. A hierarchical state machine with an Event Service Framework was used in order to create a structured control system that allowed the robot to transition between behaviors such as tape following, navigating to the shooting zone, and aiming at the target. Sensor inputs from the line-following system, bumper detection, and targeting system were continuously processed through event checkers, ensuring the robot could respond dynamically to changes in its environment. On the hardware side, motor control, sensor integration, and actuation systems were tuned to work reliably allowing consistent movement and accurate responses. The mechanical design supported stable and proper alignment for the firing mechanism, which improved targeting accuracy. Through iterative testing and debugging, the robot was refined to improve consistency and reduce unexpected behavior during multiple runs.

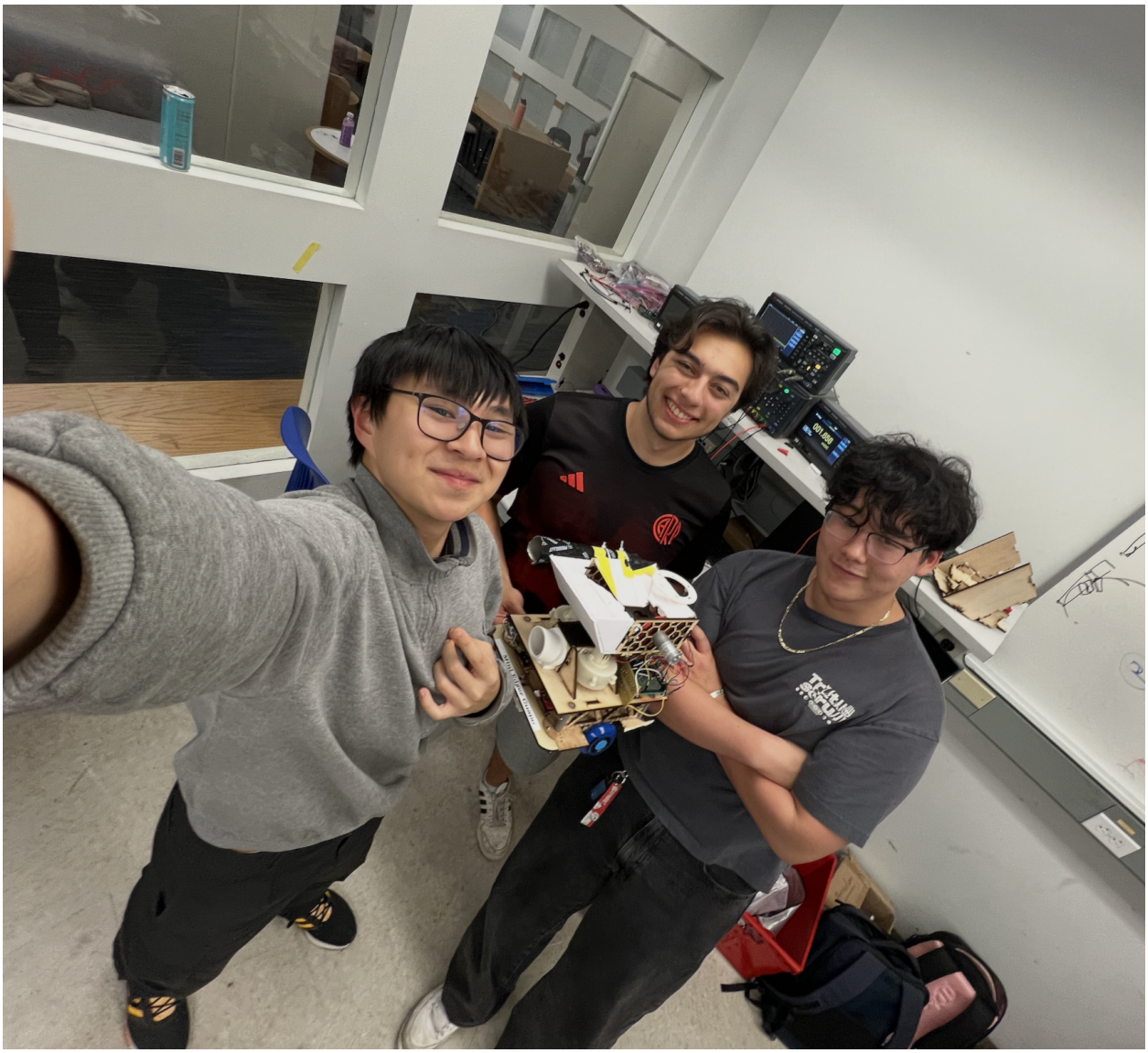


Figure 12: Team Photo: 5am on 6/5/26 after checkoff